Zephyr Project

IT-Schulungen.com
Technology & Transformation

Praxis 6
Interaktiver Event Service

Shell, GPIO-Emulation, Callback und Verarbeitung zu einer prüfbaren Anwendung verbinden

3.12

Zielarchitektur des Event Service

01	`Shell und Stimulusadapter`	validieren Kommandos und starten Pegel, Puls oder begrenzten Burst
02	`Eingabegerät und Callback`	erfassen Rohereignisse und signalisieren nicht blockierend
03	`Event Service`	verarbeitet außerhalb des Callbacks, zählt und steuert optional einen Statusausgang

event status und event stats machen Zustand, Ergebnis und Fehler vollständig sichtbar.

Arbeitsauftrag in fünf Schritten

Selbstständige Integration mit vorgegebenen Abnahmeszenarien

01

Basis nachweisen

ready, idle und definierte Zähler

02

Stimuli ausführen

Einzelpuls, zwei Pulse, Burst

03

Ergebnisse prüfen

raw, processed und rejected

04

Fehler provozieren

parallel, ungültig, nicht unterstützt

05

Abnahme sichern

Skript und Invarianten dokumentieren

Der vollständige Quellcode folgt separat; die Folien zeigen nur Auftrag, Prüfpunkte und Sollzustände.

Startzustand mit event status prüfen

ERWARTETE AUSGABE

```
event status

device: ready
sequence: idle
last_error: 0
last_event: none
raw=0 processed=0 rejected=0
```

01 Gerätebereit

Eingabegerät und native_sim-Stimulusadapter sind verfügbar.

02 Definierte Basis

Keine Sequenz läuft; Fehler, Ereigniszeit und Zähler besitzen klare Startwerte.

Die Abnahme beginnt mit einem protokollierten und reproduzierbaren Ausgangszustand.

Wiederholbares Shell-Abnahmeskript

KOMMANDOSEQUENZ

```
event status

event pulse 20
event status

event pulse 20
event status
event pulse 20
event status

event burst 4 50
event stats
```

01 Einzelszenario

Ein Puls erzeugt die vorab festgelegte Änderung der Ereigniszähler.

02 Kombiszenario

Zwei getrennte Pulse und ein begrenzter Burst bleiben einzeln nachvollziehbar.

Zwischenstände trennen die Szenarien und machen Abweichungen sofort lokalisierbar.

Zähler und prüfbare Invarianten

ROHEREIGNISSE

raw_count

steigt im Callback

beschreibt Eingangsaktivität

bleibt von Fachlogik getrennt

VERARBEITET

processed_count

steigt im Event Service

optional über Message Queue

kann Statusausgang ändern

ABGELEHNT

rejected_count

ungültig, busy, unsupported

ändert keine Ereignisbilanz

macht Bedienfehler sichtbar

Invariante: Jede Zähleränderung besitzt eine dokumentierte Ursache im Abnahmeskript.

Parallelität und Fehlerfälle

01	`Sequenz läuft`	event status zeigt aktiv; konkurrierender pulse oder burst erhält den definierten Konfliktfehler
02	`Parameter ungültig`	Text, Vorzeichen, Überlauf und Bereichsfehler ändern weder Sequenz noch Ereigniszähler
03	`Zustand nicht unterstützt`	Stimulusadapter meldet die Funktion klar als nicht verfügbar und speichert last_error

Kein Fehlerfall darf abstürzen, blockieren oder einen teilweise gestarteten Ablauf hinterlassen.

Abnahmematrix für native_sim

Startzustand	Gerät ready, Sequenz idle, Fehler und Zähler auf definiertem Ausgangswert
Einzelpuls	vollständige aktive Flanke; erwartete Änderung in raw und processed
Zwei Pulse	getrennte Anforderungen bleiben in Status und Zählerbilanz unterscheidbar
Begrenzter Burst	Anzahl und Abstand führen zur vorab berechneten Ereignisbilanz
Fehler und Parallelität	rejected steigt; Sequenz- und Ereigniszustand bleiben konsistent

Optional: Eventdaten über Message Queue übergeben und einen emulierten Statusausgang aktualisieren.

Abnahme des interaktiven Event Service

INTEGRATION

Kette vollständig

Shell steuert Stimuli

Callback bleibt minimal

Event Service verarbeitet

Statusausgang optional

BEOBACHTBARKEIT

Zustand erklärbar

Startzustand dokumentiert

letzter Fehler sichtbar

Ereigniszeit bekannt

Zähler getrennt

ROBUSTHEIT

Alle Fälle definiert

Einzelpuls

zwei getrennte Pulse

begrenzter Burst

Parallel- und Fehlerfälle

Bestanden: Stimuli und Ergebnisse sind reproduzierbar; Ausführungskontexte bleiben getrennt.